Correlation Electromagnetic Analysis on an FPGA Implementation of CRYSTALS-Kyber

Rafael Carrera Rodriguez[1][0000−0003−4779−6635], Florent Bruguier[1][0000−0002−7897−5700], Emanuele Valea[2][0000−0001−9804−7250], and Pascal Benoit[1][0000−0002−2945−5725]

[1] LIRMM, University of Montpellier, CNRS, Montpellier, France
`firstname.last-names@lirmm.fr`
[2] Univ. Grenoble Alpes, CEA, List, F-38000 Grenoble, France
`emanuele.valea@cea.fr`

Abstract. Post-quantum cryptography represents a category of cryptosystems resistant to quantum algorithms. Recently, NIST launched a process to standardize one or more of such algorithms in the key encapsulation mechanism and signature categories. Such schemes are under the scrutiny of their mathematical security, but they are not side-channel secure at the algorithm level. That is why their side-channel vulnerabilities must be assessed by the research community. In this paper, we present a non-profiled correlation electromagnetic analysis against an FPGA implementation of the chosen NIST key-encapsulation mechanism standard, CRYSTALS-Kyber. The attack correlates an electromagnetic radiation model of the polynomial multiplication execution with the captured traces. With 166,620 traces, this attack correctly recovers 100% of the subkeys. Furthermore, a countermeasure is presented for securing the target implementation against the presented attack.

Keywords: side-channel analysis · correlation power analysis · lattice-based cryptography · FPGA

1 Introduction

The advent of quantum computing represents a change of paradigm from classical algorithms. Its properties like superposition and entanglement allow for quantum algorithms, like Shor's algorithm [22], to speed up certain problems that are difficult for classic computing. In particular, this algorithm allows to break certain number theory problems in subexponential time, such as integer factorization and discrete logarithm. Many widely used asymmetric cryptosystems today, such as RSA or ECC, rely on these problems. Therefore, quantum computing poses a future threat for these systems.

This is why the National Institute of Standards and Technologies (NIST) launched an effort to choose the next cryptosystems that are resistant to quantum computing, called the Post-Quantum Cryptography (PQC) Standardization

Process [14]. The scheme chosen as the standard in the key-encapsulation mechanism (KEM) category is CRYSTALS-Kyber [2], having a mathematical primitive using lattice-based cryptography (LBC).

At first, researchers put their focus on securing the algorithms against mathematical attacks. However, the algorithms are implemented in a physical device and quantities such as power consumption, timing and electromagnetic radiation can be used to perform side-channel analyses (SCA) and obtain sensitive information, namely the plaintext or the secret key. Thus, there is an interest from the scientific community to protect the proposed PQC algorithms against such attacks [3].

Related works. Since the opening of the NIST PQC standardization effort, many SCA against LBC schemes have been presented in the literature. However, most of them are either profiling attacks [9,18,17] or attacks against the Chosen-Ciphertext-Attack (CCA) security [5,15,19,20,23], by instantiating a plaintext-checking oracle or other side-channel assisted oracles. Other works, such as [1,25], perform simple power analysis to partially or totally retrieve the secret key of the KEM. Some of these works assume an attacker model with increased capabilities, such as being able to communicate directly with the attacked server. The works from [5,9,15,19,20,23,25] create invalid ciphertexts, either from small modifications of valid ciphertexts or from completely crafted ciphertexts, in order to induce to decryption errors or to obtain decrypted plaintexts that depend directly on parts of the secret key that will have an effect on the power consumption/electromagnetic radiation.

In [11], the authors perform an unprofiled correlation power analysis (CPA) on an ARM Cortex-M4 implementation of CRYSTALS-Kyber, targeting the polynomial multiplication. In [10], the authors identify the vulnerability of this same operation within the reference implementation of CRYSTALS-Kyber, using a leakage assessment metric called the normalized inter-class variance (NICV) [4]. Recently, the authors of [13], extended this analysis of the polynomial multiplication to all the lattice-based KEMs finalists, targeting their implementations in the `pqm4` library for ARM Cortex-M4. However, all of these attacks and analyses target a software implementation.

Contribution. To the best of the authors knowledge, there are no unprofiled attacks against hardware implementations of CRYSTALS-Kyber in the literature. In this work, such attack is presented against one of the literature reconfigurable hardware (FPGA) implementations [24], using a power model that leverages the attacker's knowledge of the device's algorithm. A cost-effective countermeasure, specific to this implementation and to this attack is also proposed.

Outline. This paper is organized as follows: Section 2 presents the background necessary for this paper, including notation, a brief description of CRYSTALS-Kyber and a presentation of correlation power/electromagnetic analysis. Section

3 presents the outline of the proposed attack. Section 4 exhibits the setup and practical results of the attack. In Section 5, a simple countermeasure is presented. Finally, in Section 6, conclusions are drawn.

2 Background

2.1 Notation

The ring of integers modulo q is denoted as $\mathbb{Z}_q$. The polynomial ring defined over the previously mentioned ring, $\mathbb{Z}_q(x)= \ (x)$, is represented as R_q, where $\ (x)$ is the reduction polynomial. When a polynomial f is defined as an element of such ring, the i-th coefficient of such polynomial is denoted as f_i. The representation of such polynomial in the number theoretic transform domain is denoted as $\hat{f}$.

A vector is denoted with bold letters, *e.g.*, $\mathbf{a}$. When referring to a matrix, uppercase letters will be used, *e.g.*, A. When indexing elements of vectors or matrices, brackets will be used, *e.g.*, $A[i; j]$. When referring to all rows (resp. columns) of a matrix, a point will be used when indexing, *e.g.*, $A[\ ; j]$ (resp. $A[j;\ ]$).

2.2 CRYSTALS-Kyber

CRYSTALS-Kyber [2] is a key encapsulation mechanism (KEM) composed of two basic parts:

- An indistinguishable under chosen-plaintext attack (IND-CPA) public key encryption scheme, with its security relying on the hardness of breaking the Module Learning with Errors problem (M-LWE)[12,21].
- A Fujisaki-Okamoto transform [8], in order to convert it to an indistinguishable under chosen-ciphertext attack (IND-CCA) KEM, by re-encrypting in decapsulation and checking if the output is equal to the sent ciphertext.

The last version of CRYSTALS-Kyber uses polynomials in the ring R_q, where $q = 3329$ and the reduction polynomial is a cyclotomic polynomial $\ (x) = x^{256} + 1$. The flexible parameter is k, which is the rank of the module lattice in this cryptosystem and it dictates the size of vectors and matrices, as well as the value of other subparameters.

In CRYSTALS-Kyber, in order to reduce complexity for the polynomial multiplications, the authors choose to use the Number Theoretic Transform (NTT), which works like a Fast-Fourier Transform over the integers ring. However, because of the modulus q chosen in CRYSTALS-Kyber, the field $\mathbb{Z}_q$ contains n-th primitive roots of unity, but not $2n$-th primitive ones. This means that the defining polynomial of CRYSTALS-Kyber cannot be factorized in 256 polynomials of degree 1, but in 128 polynomials of degree 2. Therefore, the NTT of a polynomial $f\ 2\ R_q$ is a vector of 128 polynomials of degree 1. Then, according to the specification, for $\ = 17$, as the first 256-th primitive root of unity and $\mathrm{br}_7(\ )$ as the 7-bit-reversion operation, NTT is defined as follows:

$$\hat{f}_{2i} = \sum_{j=0}^{127} f_{2j}\,\zeta^{(2\mathrm{br}_7(i)+1)j}$$

$$\hat{f}_{2i+1} = \sum_{j=0}^{127} f_{2j+1}\,\zeta^{(2\mathrm{br}_7(i)+1)j}$$

that defines the polynomial:

$$\mathrm{NTT}(f) = \hat{f} = \hat{f}_0 + \hat{f}_1X + \cdots + \hat{f}_{255}X^{255}$$

As mentioned before, the actual NTT of $f \in R_q$ is a vector of polynomials. However, the authors of the cryptosystem chose to represent the result of NTT as a polynomial in R_q, even though it does not have that algebraic meaning.

NTT in Kyber can be splitted, separating the polynomial f with degree 256 and transform it into two polynomials of degree 128, where the coefficients of the first polynomial are the coefficients with even index of polynomial f, and the coefficients of the second polynomial are the coefficients with odd index of f. Then, NTT is executed for both polynomials obtaining two vectors: $(\hat{f}_0, \hat{f}_2, \hat{f}_4, \ldots, \hat{f}_{254})$ and $(\hat{f}_1, \hat{f}_3, \hat{f}_5, \ldots, \hat{f}_{255})$. Finally, the coefficients are reorganized to obtain the resulting polynomial $\hat{f}$.

Multiplication of two elements $f, g \in R_q$ is a polynomial multiplication. This operation, transformed in the NTT domain, becomes a point-wise multiplication (PWM), which, for this specific way of executing NTT, is defined as follows:

$$\hat{f} \circ \hat{g} = \hat{h} = \hat{h}_0 + \hat{h}_1X + \cdots + \hat{h}_{255}X^{255}$$

where:

$$\begin{aligned} \hat{h}_{2i} &= \hat{f}_{2i}\hat{g}_{2i} + \hat{f}_{2i+1}\hat{g}_{2i+1}\,\zeta^{2br_7(i)+1} \\ \hat{h}_{2i+1} &= \hat{f}_{2i}\hat{g}_{2i+1} + \hat{f}_{2i+1}\hat{g}_{2i} \end{aligned} \tag{1}$$

The previously described operation is executed in the CRYSTALS-Kyber scheme at the decapsulation/decryption routine with the secret key and part of the ciphertext as inputs. Therefore, this is a critical operation that can be the object of side-channel attacks, as explained in Sections 2.3 and 3.

2.3 Correlation Power/Electromagnetic Analysis

A correlation power analysis (CPA) [7] and its analog, correlation electromagnetic analysis (CEMA), is a divide-and-conquer type of attack that targets an operation with a known value and a part of the secret key, using a leakage model. The steps of such an analysis are the following:

1. The attacker retrieves n traces $t_0, t_1, \ldots, t_{n-1}$ of m sample points corresponding to the execution of a targeted operation, with $i_0, i_1, \ldots, i_{n-1}$ known inputs. For simplicity, the set of traces will be referred as the matrix T of size $n \times m$, where each row is the trace of an execution.

2. The attacker targets an unknown subkey s_0 and makes a guess g_0 from the set of l possible values of s_0.
3. According to his knowledge of the device, the attacker uses a power consumption model denoted as $M(i_j; g_0)$, depending on a certain known input i_j, for each point k of the trace $T[j; \ ]$. Then, the attacker will calculate the vector h_0 of n points, where $h_0 = M(i_j; g_o)$ for each input given to the device. If the consumption is assumed to be dependent on the set bits of the result of a targeted operation $z = F(i_j; s_0)$, then the used model will be the Hamming weight, where the consumption is modeled as $a \quad HW(z) + B$, where a is a scaling factor and B is random noise. On the other hand, if the consumption is assumed to be dependent on the switched bits from a reference state r to the result $z = F(i_j; s_0)$, then the used model is the Hamming distance, modeled as $a \ HW(z \quad r) + B$, being a bit-wise XOR. In practice, the Hamming weight model is more suitable for software implementations, whereas the Hamming distance model is more adapted to hardware implementations, with known switching activity [16].
4. The attacker uses the Pearson's correlation coefficient

$$(X; Y) = \frac{cov(X; Y)}{X \quad Y}$$

to obtain a vector c_0 of size m, where each point is obtained as:

$$(T[\ ; j]^T; h_0)$$

and $T[\ ; j]$ is equal to the column vector $(T[0; j]; T[1; j]; : : : ; T[n \quad 1; j])$. That is, the vector c_0 contains the correlation coefficient of each of the sample points with the model vector h_0.
5. The attacker repeats steps 2-4 for guesses $g_1; g_2; : : : ; g_{l-1}$, obtaining the vectors $c_1; c_2; : : : ; c_{l-1}$.
6. The attacker uses a strategy to choose the most likely guess. For example, the attacker can choose the guess g_j that yields the vector c_j that contains the point with the maximum absolute value.
7. The attacker repeats steps 2-5 for the rest of unknown subkeys until the whole key is retrieved.

3 Attack outline

3.1 Attack scenario and target

This attack targets the server part of the compact FPGA implementation of the third round NIST submission of CRYSTALS-Kyber [2], proposed by Xing and Li [24]. It was designed to fit the smallest of the Xilinx Artix-7 devices, with a resource utilization of 7412 LUTs, 4644 flip-flops, 2126 slices, 2 DSPs and 3 BRAMs. Apart from the Keccak core for hash and extendable output functions, most of the operations are done in the NTT core. This core contains two parallel and pipelined butterfly units used for NTT and inverse NTT, as well as for other operations, such as PWM, decoding, encoding, compressing and decompressing.

Apart from the attacked server device, the other two participants of this setting are a client and an adversary. The server owns a set of generated keys for executing CRYSTALS-Kyber: a private one and a public one. This keys are not regenerated. When the client wants to establish a shared key with the server for a secure communication, they do it using the encapsulation function with the public key of the server, as well as with the ciphertext to be sent. Then, the server decapsulates the shared key with their secret key, and a secure communication with symmetric cryptography can be established. The client (or other clients) repeats the process for creating other shared keys for other sessions.

In this setting, the adversary has access to the channel, capturing the ciphertexts that are being sent through it, along with the messages encrypted with the shared key. Therefore, they desire to retrieve the private key of the server, because it allows them to decapsulate the ciphertexts and decrypt all messages exchanged between the server and the client. However, the adversary is not able to communicate directly with the attacked devices. Therefore, they cannot craft special ciphertexts to attack the Fujisaki-Okamoto transform, as in [20,23].

The adversary also has physical access to the device where the server is being executed. There, they can place an electromagnetic probe and capture the radiation corresponding to the execution of the CRYSTALS-Kyber decapsulation.

3.2 Identifying vulnerable operation

In order to execute CEMA in CRYSTALS-Kyber, an operation that uses a controlled value and a part of the secret key must be chosen in the protocol. Inside the decapsulation algorithm, CRYSTALS-Kyber executes a PKE decryption, before performing the Fujisaki-Okamoto transform. This decryption is done with the secret key of the server and the ciphertext sent by the client. Going deeper, Algorithm 1 describes such decryption.

In such algorithm, the encoding operation is used to serialize byte arrays, polynomials and vectors of polynomials. The compressing operation allows to reduce the size of ciphertexts without having much effect on the correctness of the scheme. The decoding and decompressing operations are the inverses of encoding and compressing, respectively. In Line 4, an inner product between the secret key and part of the ciphertext is done. As they are in the NTT domain, each polynomial multiplication is a point-wise multiplication. This operation is then vulnerable to a CEMA as described before.

In the FPGA implementation from Xing and Li[24], PWM is done k times, according to the parameter k of CRYSTALS-Kyber, which defines the security level of the protocol and the sizes of the vectors $\mathbf{s}$ and $\mathbf{u}$. The result of each PWM is accumulated, in order to complete the matrix multiplication between $\hat{\mathbf{s}}$ and $\hat{\mathbf{u}}$. The implementation's designers allocate the PWM operation described

Algorithm 1 KYBER.CPAPKE.Dec()

```
Require: Secret key $sk \in \mathcal{B}^{12 \cdot k \cdot n/8}$
Require: Ciphertext $c \in \mathcal{B}^{d_u \cdot k \cdot n/8 + d_v \cdot n/8}$
Ensure: Message $m \in \mathcal{B}^{32}$
$\mathbf{u} := \text{Decompress}_q(\text{Decode}_{d_u}(c), d_u)$
$v := \text{Decompress}_q(\text{Decode}_{d_v}(c + d_u \cdot k \cdot n/8), d_v)$
$\hat{\mathbf{s}} := \text{Decode}_{12}(sk)$
$m := \text{Encode}_1(Compress_q(v - \text{NTT}^{-1}(\hat{\mathbf{s}}^T \circ \text{NTT}(\mathbf{u})), 1))$
                ▷ Secret key multiplied with known value. Thus, vulnerable to CEMA
return $m$
```

in Equation 1 in two clock cycles: PW0 and PW1, shown in (2).

$$\text{PWM0: } n_0 = \hat{f}_{2i} + \hat{f}_{2i+1};\ n_1 = \hat{g}_{2i} + \hat{g}_{2i+1};\ m_0 = \hat{f}_{2i} \quad \hat{g}_{2i};\ m_1 = \hat{f}_{2i+1} \quad \hat{g}_{2i+1}$$
$$\text{PWM1: } n_2 = m_0 + m_1;\ m_2 = n_0 \quad n_1;\ m_3 = m_1 \quad {}^{2br(i)+1};$$
$$\hat{h}_{2i} = m_0 + m_3;\ \hat{h}_{2i+1} = m_2 \quad n_2 \tag{2}$$

In the case of PWM between two polynomials in the secret vector $\hat{\mathbf{s}}$ and the ciphertext $\hat{\mathbf{u}}$, the inputs are $\hat{f} = \hat{\mathbf{s}}[j]$ and $\hat{g} = \hat{\mathbf{u}}[j]$ (or viceversa), for an integer $j\ 2\ [0; k \quad 1]$. Now, the first cycle, *i.e.*, PWM0, is of special importance because it executes the operations $m_0 = \hat{\mathbf{s}}[j]_{2i} \quad \hat{\mathbf{u}}[j]_{2i}$ and $m_1 = \hat{\mathbf{s}}[j]_{2i+1} \quad \hat{\mathbf{u}}[j]_{2i+1}$. These two operations are done between a controlled input by the client and the secret key of the server. Thus, it is the final internal target for the CEMA in this work.

However, both targeted operations are executed at the same time, in one single clock cycle. To circumvent this issue, one can use the same traces for two different analyses. In the first analysis, the attacker can execute the CEMA procedure and obtain a guess for $\hat{\mathbf{s}}[j]_{2i}$, assuming that the other operations of the device, including the multiplication $\hat{\mathbf{s}}[j]_{2i+1} \quad \hat{\mathbf{u}}[j]_{2i+1}$, generate random noise in the power trace that does not affect the correlation analysis. For the second analysis, the attacker uses the same power traces and obtains a guess for $\hat{\mathbf{s}}[j]_{2i+1}$, making the same noise assumption in the first analysis.

3.3 Usage of Hamming distance as consumption model

As the target is a hardware implementation instead of a software one, a more precise model is needed. Therefore, the decision was made to use the Hamming distance model, which requires the knowledge of previous reference values of an operation. In this setting, those reference values can be known by the attacker, because of their knowledge of the algorithm implementation.

In the implementation from Xing and Li [24], the reference values $r[j \quad 256]; r[j \quad 256 + 1]$ for $j\ 2\ [0; k \quad 1]$ can be known for sure. These values are the result of the physical multipliers before modular reduction and before starting the first PWM0 cycle for each PWM. The operation before such cycles is the ending of the NTT of $\hat{\mathbf{u}}[j]$. Therefore, r_{j*256} is the result of the last multiplication in the

NTT algorithm of the even values of $\hat{u}[j]$ and $r[j\ \ 256+1]$ corresponds to the result of the last multiplication for the NTT of the odd values of $\hat{u}[j]$. Then, for obtaining such result, it suffices to revert the last steps used to obtain $\hat{u}[j]$. Such reversal is presented in (3). The reader can refer to [24] for details on the used NTT algorithm.

$$
\begin{aligned}
r[j\ \ 256] &= (((\hat{u}[j]_{252}\ \ \hat{u}[j]_{254})\ \ \mathrm{two}' \to {}') \bmod q) \to \\
r[j\ \ 256+1] &= (((\hat{u}[j]_{253}\ \ \hat{u}[j]_{255})\ \ \mathrm{two}' \to {}') \bmod q) \to
\end{aligned}
\tag{3}
$$

The rest of the reference values depend on the success of the attack on the previous subkeys. They correspond to the previous result m_0 for the even values and to m_1 for the odd values in cycle PWM0 in (2). Such reference values are obtained like in (4), for $i \in [0;127]$.

$$
\begin{aligned}
r[j\ \ 256+2i] &= \hat{u}[j]_{2i}\ \ \hat{s}[j]_{2i} \\
r[j\ \ 256+2i+1] &= \hat{u}[j]_{2i+1}\ \ \hat{s}[j]_{2i+1}
\end{aligned}
\tag{4}
$$

After executing the acquisition campaign, the analysis phase must be carried out sequentially, obtaining at first the subkeys $\hat{s}[j]_0$ and $\hat{s}[j]_1$, in order to obtain the next reference values for establishing the Hamming distance model.

4 Results

4.1 Attack setup

An FPGA evaluation general-purpose board was programmed with the implementation from [24]. The FPGA is configured with the target implementation slightly modified. The modifications consist of a UART interface for communication with the computer and a trigger to start the capture of the traces on the desired PWM operation. The evaluation board is a Digilent Basys-3 with a Xilinx Artix-7 XC7A35TCPG236 FPGA chip. The signal is shaped by a Langer RF-U 5-2 EM probe coupled with a Femto HSA-X-2-40 amplifier and captured with a Tektronix MSO64 oscilloscope configured to use a low-pass filter with a cut-off frequency of 200 MHz.

The security parameter k from CRYSTALS-Kyber was set to 2, the lowest one, for ease of analysis. It should be noted that increasing the security parameter should not make the attack more difficult. The effect of such a modification is to increase the number of subkeys, but because of the divide-and-conquer nature of this attack, this would only lead to a linear increase of the attack runtime.

The sample rate of the oscilloscope was fixed to 1.25 GS/s, whereas the FPGA was programmed to use a clock of 62.5 MHz generated by the Artix-7 chip which in turn, gets its 100 MHz reference clock from the Basys-3 board. Therefore, the number of samples per clock cycle is 20. 15 sets of about 11k traces are used (166,620 traces in total), calculating the correlation incrementally with each set.